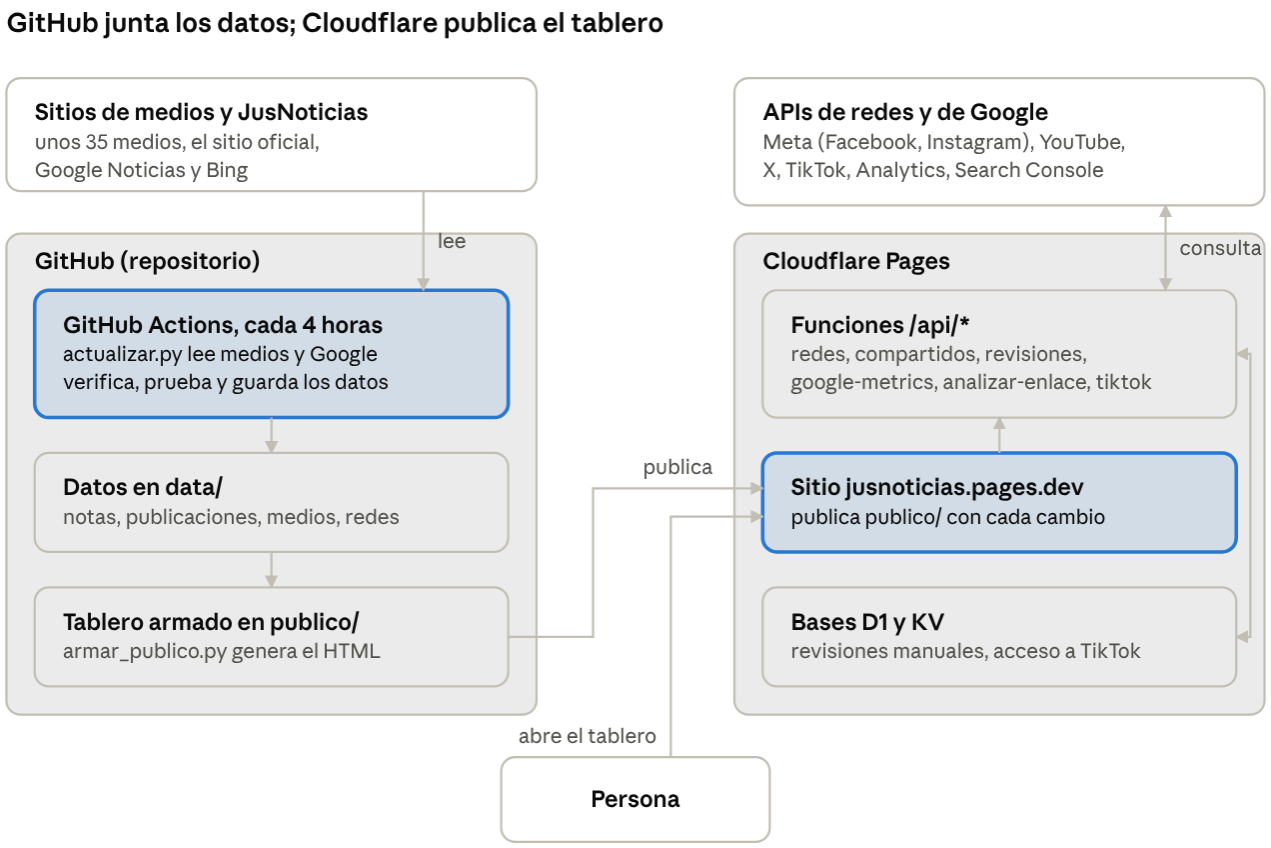
Tablero JusNoticias: arquitectura técnica

Oct 26, 2026 · @Ignacio

Vista general

El sistema tiene dos partes: **GitHub** junta y guarda los datos, y **Cloudflare Pages** publica el tablero y atiende las consultas en vivo. No hay servidor propio ni base de datos central: los datos son archivos JSON dentro del repositorio `ACJ2026/redes-jusnoticias`.



arquitectura · 2 plataformas, 3 grupos de fuentes

Cada 4 horas GitHub Actions lee medios y Google, guarda los JSON y arma el HTML; Cloudflare publica cada cambio de `main`. Lo que se consulta en el momento (redes, Google Analytics, revisiones) pasa por las funciones `/api/*` de Cloudflare.

Estructura del repositorio

Todo vive en `ACJ2026/redes-jusnoticias` (público desde el 26/10/2026; antes privado). La rama de producción es `main`.

Ruta	Función
`scripts/actualizar.py`	Recolector principal: notas, medios, Google Noticias, comparación de textos
`scripts/verificar_publicaciones.py`	Abre las publicaciones dudosas y las confirma, descarta o deja pendientes
`scripts/historial.py`	Normalización de URL, deduplicación, conservación del historial, revisiones manuales, estado de medios
`scripts/verificar_historial.py`	Control que frena el guardado si falta una publicación de una nota vigente
`scripts/armar_publico.py`	Arma el HTML público con la plantilla y los datos
`scripts/armar_inventario.py`	Arma el inventario único de medios
`scripts/descubrir_redes.py`	Busca las cuentas en redes en la portada de cada medio (solo informa)
`scripts/fuentes.py`	Funciones auxiliares de lectura de fuentes
`templates/tablero.html`	Plantilla del tablero: HTML, CSS y JavaScript en un solo archivo
`publico/`	Lo que publica Cloudflare: `index.html`, `tablero_publico.html`, `privacidad.html`, `terminos.html` y archivos de verificación de TikTok
`functions/api/`	Funciones de Cloudflare Pages (APIs)
`api/`	Versiones viejas para Vercel (`redes.js`, `google-metrics.js`); no se usan en producción
`data/`	Datos en JSON
`migrations/`	SQL de las bases D1 (`0001` TikTok, `0002` revisiones)
`tests/`	Pruebas en Python y Node
`.github/workflows/`	7 flujos de GitHub Actions

Ruta	Función
`tablero_interno/index.html`	Copia del tablero interno que vivía como artefacto de Claude
`docs/`	Manual del tablero, Cloudflare, Google, redes, TikTok y auditoría de cobertura
`BITACORA.md`, `REGLAS.md`, `CONTINUIDAD_IA.md`, `README.md`	Registro de cambios, reglas de trabajo, traspaso entre IAs y descripción general
`vercel.json`	Configuración heredada de Vercel (salida `publico`)
`infografias_borrador/`, `antes.html`	Material de trabajo; no se publica

Relevamiento automático

`scripts/actualizar.py out data/rep2.json` corre en GitHub Actions cada 4 horas y escribe en `out/` los JSON nuevos. Sigue estos pasos en orden:

1. **Notas oficiales:** lee las notas de jusnoticias.juschubut.gov.ar por la API de WordPress (id, fecha, título, tipo).
2. **Medios con WordPress:** lee las últimas publicaciones de cada sitio por su API («lectura directa»). Detecta solos los sitios WordPress nuevos.
3. **Medios con índices propios:** sitemaps mensuales (ADN Sur, EQS Notas), sitemap de texto (La Opinión Austral), índice por número (LU17, El Extremo Sur, La Vanguardia), sitemaps comprimidos (ABC Diario) y La Tecla Patagonia.
4. **Comparación por título:** `match()` compara títulos por fragmentos de palabras (umbral 0,57), en una ventana de un día antes a 8 días después de la nota.
5. **Segunda oportunidad por texto:** `cerca()` junta candidatas con título parecido y `confirmar_por_texto()` las confirma solo con evidencia fuerte en el cuerpo.
6. **Lectura de texto completo:** lee el texto de las publicaciones de los últimos 10 días que todavía no leyó (lista en `textos_revisados.json`). Clasifica: copia textual (50 % o más igual), reescritura o redacción propia, y si cita a Juschubut Noticias u otro medio.
7. **Coincidencia por datos:** un puntaje que pesa las palabras raras (IDF). Desde 0,25 entra «A verificar» con su `motivo`. Si una nota junta más de 4 candidatas, se la trata como genérica y se omite.
8. **Google Noticias y Bing:** busca cada nota por su título y por dos frases textuales. Un medio fuera de la lista solo entra si menciona Chubut o una ciudad de la provincia. Hace seguimiento de dominios descubiertos (`fuentes_descubiertas.json`).

9. **Medios que fallan:** si un medio no responde, se conservan sus filas anteriores («error aislado») y se registra en `estado_medios.json`.
10. **Historial y revisiones:** vuelve a sumar las publicaciones históricas de notas vigentes, aplica las decisiones manuales de `revision_manual.json` y deduplica por URL normalizada.
11. **Verificación de candidatas:** después, `verificar_publicaciones.py` abre cada candidata dudosa, compara título y cuerpo con la nota oficial y la confirma (confianza Alta), la descarta (Baja) o la deja pendiente. Si el medio redirige, guarda la URL vieja en `url_alternativas`.

El tablero cuenta como «verificada» una fila con confianza distinta de Baja, que no sea solo por título, que no sea de Google Noticias y que tenga URL directa.

Archivos de datos

Todos están en `data/` y se guardan con cada actualización. Al 26/10/2026 hay 284 notas y unas 1.487 publicaciones.

Archivo	Qué guarda	Quién lo escribe
`notas2.json`	Notas oficiales: id, fecha, título, tipo, mes	actualizar.py
`rep2.json`	Publicaciones encontradas: nota_id, medio, origen, cita_fuente_oficial, cita_otro_medio, confianza, metodo, url, motivo, url_alternativas, key	actualizar.py y verificar_publicaciones.py
`medios.json`	Cobertura por medio (qué se revisó y cómo)	actualizar.py
`medios_inventario.json`	Un registro por medio: dominio, tipo, localidad, método, estado, publicaciones, redes	armar_inventario.py (los campos manuales no se pisan)
`estado_medios.json`	Última revisión correcta, último fallo, fallos seguidos y alerta de cada medio	actualizar.py

Archivo	Qué guarda	Quién lo escribe
`textos_revisados.json`	URLs ya leídas en texto completo (más de 10.000)	actualizar.py
`fuentes_descubiertas.json`	Dominios encontrados por buscadores para seguimiento	actualizar.py
`revision_manual.json`	Decisiones manuales cargadas en el repositorio	A mano
`redes2.json`	Historial de publicaciones de redes (propias y de terceros) con métricas	Carga histórica
`canales.json`	Inventario de nuestras cuentas: red, cuenta, seguidores, estado	A mano
`cuentas_monitoreo.json`	Cuentas de medios en cada red que se vigilan (332 filas; 64 de Instagram activas)	A mano y con descubrir_redes.py
`actualizado.txt`	Fecha y hora UTC del último relevamiento	Flujo de actualización

Regla: la clave `key` (`p0`, `p1`…) se renumera en cada corrida; nunca se usa para identificar una fila. Una fila se identifica por nota, medio y URL normalizada.

Flujos de GitHub Actions

Hay 7 flujos en `.github/workflows/`. El único que guarda datos es `actualizar.yml`.

Flujo	Cuándo corre	Qué hace
`actualizar.yml` «Actualizar tablero»	Cada 4 horas (cron UTC `59 0,4,8,12,16,20`); al cambiar el recolector o el verificador en `main` ; a mano	Recolecta, verifica, controla el historial, arma el inventario, registra la fecha, arma el HTML, corre las pruebas y guarda. Modo `publico` a mano: solo rearma el HTML
`reconstruir-publico.yml`	Al cambiar la plantilla, `armar_publico.py` , las pruebas o los datos en `main` ; a mano	Regenera los dos HTML públicos, valida el JavaScript, corre las pruebas y guarda el HTML
`validar-tablero.yml` «Validar tablero y fuentes de tráfico»	En cada PR a `main` y en cada cambio de `main`	Revisa la sintaxis de todas las funciones, prueba las APIs de redes y TikTok, revisa el JavaScript del tablero y corre las pruebas
`test-historial.yml` «Pruebas de regresión del historial»	En cada PR	Sintaxis del JavaScript del tablero público e interno y todas las pruebas de Python
`probar-verificador.yml`	En PR que cambian el verificador	Pruebas del verificador de publicaciones
`descubrir-redes.yml`	Solo a mano	Lee las portadas de los medios y lista sus cuentas en redes; no guarda nada
`prueba-integracion-aislada.yml`	En dos ramas viejas de prueba y a mano	Corre el recolector sin guardar, para probar el historial

Por regla de trabajo, un PR se fusiona solo cuando pasan `validar` , `pruebas-historial` y el despliegue de vista previa de Cloudflare Pages. El flujo de actualización tiene un grupo de concurrencia: no corren dos a la vez en la misma rama.

Protecciones del historial y del guardado

Una actualización nunca debe borrar publicaciones ya encontradas. Estos controles corren antes de guardar, en este orden:

1. **Conservación en el recolector:** las publicaciones de notas vigentes que una corrida no vuelve a encontrar se reincorporan, salvo falsos positivos explícitos. Se hace dos veces: antes y después de los filtros de limpieza.
2. **Control de identidad (`verificar_historial.py`):** compara la versión anterior con la nueva por nota, medio y URL normalizada (también entre `url_alternativas`). Si falta una, el flujo se detiene y no guarda.
3. **Control de volumen:** si las publicaciones bajan a menos del 80 % o las notas a menos del 95 %, no se guarda.
4. **Pruebas antes de guardar:** se compila el código, se valida el JavaScript del tablero y se corren todas las pruebas.
5. **Guardado seguro:** el commit se hace como `jusnoticias-bot`, con `git pull --rebase` antes de subir. Si otra corrida ya guardó y hay conflicto, esta se detiene sin pisar nada.

Normalización de URL: unifica http y https, saca `www.`, mayúsculas del dominio, barra final, fragmentos, parámetros de seguimiento y variantes AMP. Las de Google Noticias se conservan por su identificador.

Reglas de datos de `REGLAS.md`: no borrar filas encontradas por otro método, mismo nombre de medio siempre, no inventar resultados y fechas en hora argentina.

Generación del tablero

El tablero es una sola página HTML estática con los datos adentro. No usa frameworks ni base de datos para mostrarse.

- **Plantilla:** `templates/tablero.html` tiene el HTML, el CSS y todo el JavaScript. Los datos se inyectan en un bloque `<script type="application/json" id="datos">`.
- **Armado:** `python3 scripts/armar_publico.py data/notas2.json data/rep2.json data/redes2.json data/canales.json data/medios.json publico/tablero_publico.html` y luego se copia a `publico/index.html`. Agrega también inventario, cuentas, localidades, estado de medios y fecha de actualización. No incluye secretos.
- **Cálculos en el navegador:** con los datos, el JavaScript calcula todo según el período elegido: notas, porcentaje, publicaciones, medios por nota (verificados + a verificar, cada medio una vez), rankings, temas, localidades y la infografía (dibujada en `<canvas>`, se descarga como PNG).

- **Revisiones en vivo:** al abrir, el tablero pide `/api/revisiones` y aplica las decisiones guardadas encima de los datos.
- **Diseño:** colores por variables CSS con modo claro y oscuro; en pantallas de hasta 600 px las tablas se vuelven fichas. Cada sección tiene su botón de ayuda con el texto en un `<template>`.
- **Tablero interno:** `tablero_interno/index.html` es una copia del tablero interno que vive como artefacto de Claude; no se publica en Cloudflare.

Funciones de Cloudflare Pages y bases

Cloudflare Pages publica `publico/` desde `main` (comando de build `exit 0`) y ejecuta las funciones de `functions/`. Cada PR genera una vista previa propia.

Ruta	Método	Qué hace
`/api/redes`	GET	Métricas en vivo de nuestras cuentas: Facebook (dos páginas), Instagram, YouTube, X y TikTok. Hasta 90 días, con paginación y aviso de límite
`/api/compartidos`	GET	Repercusiones de otras cuentas. `?plan=1` devuelve las fuentes y sus tandas; `?fuente=ID&lote=N` consulta una tanda. Fuentes: cuentas de Instagram (tandas de 12), etiquetas de Instagram, búsqueda en YouTube (10 notas), canales de YouTube de medios (tandas de 20) y hashtags (apagado)
`/api/analizar-enlace`	POST	Lee un enlace de cualquier red o medio (solo HTTPS, sin direcciones internas, 3 redirecciones, 8 s) y lo compara con todas las notas
`/api/revisiones`	GET / POST	Lee y guarda decisiones manuales (confirmar o descartar). POST exige la clave, el mismo origen y URL directa del medio

Ruta	Método	Qué hace
`/api/google-metrics`	GET	Datos de Google Analytics 4 (visitas, canales, dispositivos, fuentes) y Search Console (búsquedas, páginas), hasta 90 días
`/api/tiktok`, `/api/tiktok/auth`, `/api/tiktok/callback`	GET / POST	Conexión OAuth con TikTok y lectura de la cuenta autorizada; renueva el token solo

Las tandas existen porque Cloudflare limita las consultas externas por pedido (50 en el plan gratuito); ninguna tanda pasa de 40.

Bases:

- **D1** **`jusnoticias-revisiones`** (variable `REVISIONES_DB`): tabla `revisiones_publicacion` con la última decisión por publicación y `bitacora_revisiones_publicacion` con todas las decisiones. Creada con `migrations/0002_revisiones_publicaciones.sql`.
- **D1** **`TIKTOK_AUTH_DB`** y **KV** **`TIKTOK_KV`**: estado y tokens de la conexión con TikTok (`migrations/0001_tiktok_auth_security.sql`).

Secretos y configuración

Ninguna clave está en el repositorio ni en el HTML: viven en la configuración del proyecto `jusnoticias` de Cloudflare Pages (Settings → Variables and Secrets y Bindings). Se revisó todo el historial del repositorio el 26/10/2026 antes de hacerlo público. GitHub Actions no usa secretos.

Nombre	Para qué	Lo usa
`META_TOKEN`	Acceso a Facebook e Instagram (Graph API v26)	redes, compartidos
`IG_USER_ID`	Cuenta de Instagram propia (base de `business_discovery`)	redes, compartidos
`FB_PAGE_ID`, `FB_PAGE_TOKEN` (y sus variantes por página)	Páginas JusNoticias y JusNoticias STJ	redes
`YT_KEY`, `YT_CANAL_ID`	YouTube Data API y canal propio a excluir	redes, compartidos
`X_BEARER_TOKEN`, `X_USER_HANDLE`	API de X (hoy sin créditos)	redes

Nombre	Para qué	Lo usa
`TIKTOK_CLIENT_KEY`, `TIKTOK_CLIENT_SECRET`, `TIKTOK_REDIRECT_URI`, `TIKTOK_CONNECT_KEY`	Conexión OAuth con TikTok	tiktok
`TIKTOK_AUTH_DB` (D1), `TIKTOK_KV` (KV)	Estado y tokens de TikTok	tiktok
`GOOGLE_SERVICE_ACCOUNT_JSON`, `GA4_PROPERTY_ID`, `GSC_SITE_URL`	Google Analytics 4 y Search Console	google-metrics
`REVISIONES_DB` (D1), `REVISIONES_CLAVE`	Guardado de revisiones manuales y su clave	revisiones
`IG_HASHTAGS`	Si vale `1`, activa la búsqueda por hashtags (requiere aprobación de Meta)	compartidos

La clave de revisiones se pide en el navegador la primera vez y se guarda solo durante la sesión.

Pruebas y control de calidad

Hay 106 pruebas de Python y 3 archivos de pruebas de Node. Corren en cada PR, en cada cambio de `main` y antes de cada guardado de datos.

Archivo	Qué prueba
`tests/test_tablero_publico.py`	Plantilla y HTML público: textos, botones, secciones, versión móvil, que no haya secretos, reglas de conteo, flujos de trabajo y funciones de redes
`tests/test_conservar_historial.py`	Que no se pierdan publicaciones, deduplicación y revisiones manuales
`tests/test_monitoreo_buscadores.py`	Detección por buscadores, lectura de texto, coincidencias por datos y estado de medios
`tests/test_verificar_publicaciones.py`	Verificador: enlaces de buscadores, comparación de texto, mensajes de error, conservación de la URL anterior

Archivo	Qué prueba
`tests/test_tiktok_migration.py`	Esquema de la base de TikTok
`tests/test_redes_x.mjs`, `test_redes_meta.mjs`, `test_tiktok.mjs`	APIs de X, Meta y la seguridad de TikTok

Además se revisa la sintaxis de todas las funciones y del JavaScript del tablero. Los cambios visuales se comprueban con capturas en el navegador (Playwright) a 1280, 1024, 768, 390 y 320 px.

Comando local: `python3 -m unittest discover -s tests -v`.

Operación, costos y límites

Todo funciona sin pagar, pero cada plataforma tiene un límite que condiciona el diseño.

Plataforma	Límite	Cómo se maneja
GitHub Actions	Repositorio privado: 2.000 minutos gratis por mes. En octubre se superaron y del 25/10 a la noche al 26/10 no corrió nada	Repositorio hecho público (minutos ilimitados) y relevamiento cada 4 horas. Si vuelve a privado, hacerlo cuando se renueve el cupo y pasar a 6 horas
Cloudflare Pages	50 consultas externas por pedido (plan gratuito)	Consultas de redes en tandas de hasta 40
Meta (Instagram)	Límite de consultas por hora; solo cuentas de empresa o creador; etiquetas y hashtags requieren permisos que no están	1 página (30 publicaciones) por cuenta por defecto; «Buscar ahora» como mucho una vez por hora
Meta (Facebook de terceros)	Requiere un permiso que Meta casi no otorga	Búsqueda asistida
YouTube Data API	Cuota diaria por unidades (búsqueda 100, canal 1, lista 1)	10 notas por búsqueda; canales por lista de videos (barato)
X	API de lectura paga	Registros guardados
TikTok	Solo la cuenta autorizada	Búsqueda asistida para terceros
Google Analytics / Search Console	Hasta 90 días por consulta	Se toma la ventana más reciente

Horario: relevamiento a las 21:59, 01:59, 05:59, 09:59, 13:59 y 17:59 (hora argentina). Una corrida completa tarda entre 10 y 40 minutos.

Forma de trabajo y pendientes

En el repositorio trabajan Claude y GPT/Codex, con la cuenta ACJ2026. Ignacio aprueba los cambios.

- **Cada cambio en una rama y un PR.** Los de Claude se fusionan solos cuando pasan los tres chequeos y no hay conflictos; los cambios de detección o de datos se consultan antes.
- **Antes de tocar algo:** revisar los PR abiertos y los últimos cambios de `main` para no pisar el trabajo del otro.

- **Bitácora:** cada cambio queda en `BITACORA.md` con qué, por qué, PR y resultado. `REGLAS.md` y `CONTINUIDAD_IA.md` sirven para el traspaso entre IAs.
- **Datos generados:** el HTML público se regenera con el script; nunca se edita a mano.

Pendientes técnicos al 26/10/2026:

- [ ] Ver el detalle de errores de Instagram y depurar las cuentas que fallan (35 de 64 en la primera prueba).
- [ ] Regenerar el token de Meta con permiso de etiquetas para activar «Instagram (etiquetas)».
- [ ] Decidir si se tramita la aprobación de Meta para hashtags.
- [ ] Resolver los 4 canales de YouTube que no se encuentran (direcciones `/c/`).
- [ ] Revisar los medios que no responden: El Diario Web, El Extremo Sur, LU17, Metadata Noticias, Rio Negro.
- [ ] Decidir cuándo volver el repositorio a privado y con qué frecuencia de relevamiento.
- [ ] Cerrar o adaptar el PR #88 de GPT, que contradice el criterio del total de medios, y revisar los PR #94, #104 y #106 que siguen abiertos.
- [ ] Actualizar `README.md`, `REGLAS.md` y `CONTINUIDAD_IA.md`, que todavía nombran el repositorio viejo y el horario de 2 horas.